

QE GENERATOR — OVERVIEW

# Tự động hóa **Test Script** cho ETL Mapping

Trình bày cho QE Lead · May 2026

## BỐI CẢNH

# Tại sao cần QE Generator?

## ✗ Trước đây

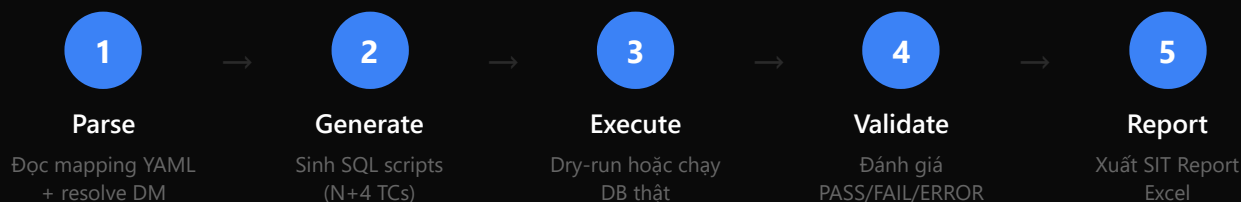
- Viết tay SQL test case cho từng bảng
- Không nhất quán giữa QE với nhau
- Mất nhiều giờ cho mỗi mapping unit
- Khó trace khi mapping thay đổi

## ✓ Với QE Generator

- Tự động sinh SQL từ mapping YAML
- Chuẩn hóa theo công thức N+4
- Chạy trong vài giây cho nhiều unit
- YAML snapshot → detect thay đổi

QUY TRÌNH

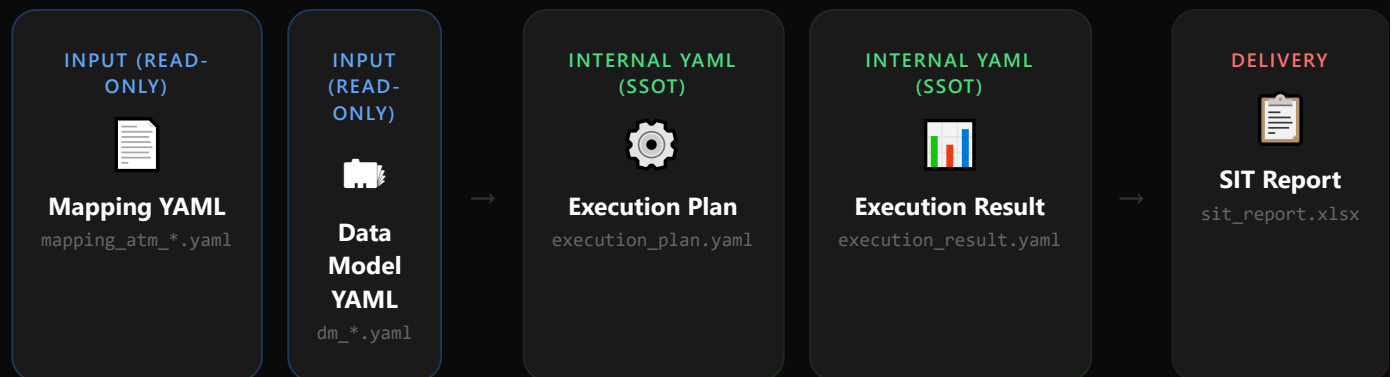
# Pipeline 5 bước



Lệnh duy nhất: `qe-generator run <mapping.yaml>`

## ARTIFACTS

# Đầu vào & Đầu ra



⚠ Excel được *generate* từ YAML — không bao giờ là source of truth

NGUYÊN TẮC SINH TC

# Công thức N + 4

N + 4

test cases cho mỗi mapping unit có N fields

| TC        | TÊN         | LAYER      | ASSERTION      | MỤC ĐÍCH                        |
|-----------|-------------|------------|----------------|---------------------------------|
| TC01      | count       | gate       | DIFF_CNT = 0   | Row count source = target       |
| TC02      | dup_nk      | gate       | DUP_CNT = 0    | Không trùng natural key         |
| TC03      | filter      | core       | ORPHAN_CNT = 0 | Record source đều vào target    |
| TC04..3+N | field_{col} | core       | DIFF_CNT = 0   | Transformation logic từng field |
| TC{4+N}   | nonnull     | supplement | NULL_CNT = 0   | Cột mandatory không chứa NULL   |

## EXECUTION ORDER

# Thứ tự chạy & Dependency

Gate phải PASS trước — nếu count sai, các TC sau vô nghĩa



## PHÂN LOẠI TRANSFORMATION

# Rule Classifier — 3 mức ưu tiên

## ① etl\_derived

Khi `transformation = null`

Giá trị do ETL hardcoded — sinh SQL placeholder, không test

## ② Named rule

Transformation chứa `{rule_id}(...)`

Lookup `transformation_rules.yaml` → lấy category  
→ map rule\_type

## ③ direct

Không match case nào

Column reference đơn giản, dùng

`field_direct.sql.j2`

## surrogate\_key

Category đặc biệt — dùng template riêng

`field_surrogate.sql.j2` — so sánh hash output

## EXECUTION MODE

# Dry-run **vs** Real Execution

## Dry-run (mặc định)

- Không cần kết nối DB
- Validate SQL syntax (parentheses, trailing comma...)
- Phát hiện placeholder chưa implement
- Output: `mode: dry_run` trong YAML

## Real Execution (`--execute`)

- Kết nối DB qua ODBC config
- Chạy theo thứ tự gate → core → supplement
- Retry 1 lần khi mất connection
- Output: kết quả thực tế với `actual_result`

Kết quả mọi mode đều lưu trong `execution_result.yaml` — cấu trúc thống nhất



## ERROR HANDLING

# 4 cấp độ xử lý lỗi

-  **Fatal** Dừng toàn bộ pipeline — thiếu field bắt buộc, etl\_handle không hợp lệ, không tìm thấy DM file
-  **Unit-Fatal** Skip unit hiện tại, tiếp tục unit khác — không có PK column, duplicate SQL filename
-  **Warning** Log cảnh báo, pipeline tiếp tục — rule\_id không tìm thấy (fallback direct), column không có trong DM
-  **Connection** Retry 1 lần — nếu thất bại mark tất cả TC còn lại là ERROR

 Done ·  Error stop ·  Status ·  Warning

## BÁO CÁO SIT

# Cấu trúc **sit\_report.xlsx**

**Sheet: SIT\_REPORT**

- STT | Tên bảng
- Passed / Failed / Not run / Tổng
- Tỷ lệ (%)
- Công thức: failed = FAIL + ERROR;  
not\_run = Skipped + Pending

**Sheet: <unit\_name>**

- Test Case ID | Chức năng | Mục đích
- SQL script | Kết quả mong đợi
- Kết quả thực tế | Trạng thái
- Ngày test | Người test | Evident

Auto backup trước khi ghi đè · Giữ tối đa 2 bản (.bak1, .bak2)

## CLI REFERENCE — MILESTONE 1

# Các lệnh thường dùng

**qe-generator parse <file>**

Validate mapping + resolve DM — kiểm tra trước khi generate

**qe-generator generate <file>**

Sinh toàn bộ SQL scripts theo N+4 formula

**qe-generator run <file>**

Full pipeline — dry-run mặc định, thêm --all cho multi-unit

**qe-generator run <file> --execute --config db.yaml**

Full pipeline + chạy thật trên DB, xuất kết quả thực tế

**qe-generator report**

Tạo lại sit\_report.xlsx từ execution\_result.yaml có sẵn

**qe-generator run <dir> --all**

Quét recursive tìm tất cả mapping\_atm\_\*.yaml (trừ rules/ và dotfiles)



## MILESTONE 2

# Milestone 1 → Milestone 2



## Milestone 1 — Done

- Parse mapping YAML + resolve DM
- Sinh N+4 TC, render SQL Jinja2
- Dry-run + Real Execution (ODBC)
- SIT Report Excel + snapshot/manifest
- Change Detection (fingerprint / hash)
- CLI: parse / generate / run / report



## Milestone 2 — Planned

- **Req 29** — TC theo Storage Pattern (SCD2/SCD4A/Fact)
- **Req 28** — Coverage Report (exit criteria)
- **Req 30** — Multi-source target scoping
- **Req 24/25** — Change History + CLI history
- **Req 17+** — CLI: diff / aggregate-report / -changed-only
- **Req 27** — Dependency Graph (--cascade)

REQ 29 · PRIORITY: CAO NHẤT

# TC theo Storage Pattern

Hiện tại: N+4 cho mọi bảng. Milestone 2: N+4+M+P — thêm TC đặc thù theo cơ chế lưu trữ

| STORAGE<br>PATTERN | SCENARIOS                               | TC THÊM<br>(M) | PRECONDITION<br>(P) | TỔNG<br>THÊM |
|--------------------|-----------------------------------------|----------------|---------------------|--------------|
| overwrite          | —                                       | 0              | 0                   | không đổi    |
| scd4a              | INSERT · UPDATE · DELETE                | 7              | 3                   | +10 TC       |
| scd2               | INSERT · UPDATE · DELETE · NO<br>CHANGE | 11             | 4                   | +15 TC       |
| fact_append        | NEW BATCH · RERUN · CROSS-<br>BATCH     | 6              | 3                   | +9 TC        |
| fact_snapshot      | DAILY · COMPLETENESS ·<br>RERUN         | 5              | 3                   | +8 TC        |

🚩 Precondition TC xác nhận data thực sự chứa scenario đó · Precondition FAIL → "No Data", KHÔNG fail TC logic

## REQ 29 · ACCEPTANCE CRITERIA — KIRO SPEC

# Storage Pattern TC — 4 AC chính

## AC-1 · Parse storage\_pattern

Parser PHẢI đọc `target.etl_handle` và map sang `storage_pattern`. Giá trị hợp lệ: `overwrite` | `scd2` | `scd4a` | `fact_append` | `fact_snapshot`. Mặc định: `overwrite`. Giá trị không hợp lệ → raise V-P5.

## AC-2 · Sinh TC theo bảng N+4+M+P

Generator PHẢI sinh đúng số TC per pattern. Mỗi TC đặc thù có field `scenario` trong metadata. Template naming: `<pattern>_<logical_id>.sql.j2`. Ví dụ: `scd4a_insert_current.sql.j2`.

## AC-3 · Precondition logic

Mỗi scenario có 1 Precondition TC (assertion: COUNT > 0). Nếu Precondition FAIL: đánh dấu scenario "No Data", set TC logic = `Skipped (No Data)`, in ⚠️ "Scenario {name} chưa có dữ liệu test — cần bổ sung data UAT".

## AC-4 · Execution Plan + Dependency

TC Precondition: `layer: scenario`. TC logic đặc thù: `layer: core`. TC logic PHẢI có `depends_on` trỏ đến Precondition TC tương ứng. Cả hai xuất hiện trong `execution_plan.yaml` và `testcase_manifest.yaml`.

REQ 28 · PRIORITY: CAO

# Coverage Report — Exit Criteria

⚠ Coverage Report là điều kiện bắt buộc để QE Lead approve gate Generate → Execute

## output/coverage\_report.xlsx — 4 sheets

- **Summary** — % Scenario Coverage, Pass Rate, Fail Rate, số TC pending review
- **Scenario Coverage** — per scenario: status, ngày chạy cuối. Highlight: vàng (Not Run), đỏ (FAIL), cam (cần review)
- **Field Coverage** — per field: rule\_type, có TC không. Highlight đỏ field không có TC
- **Review Required** — TC cần manual review, transformation gốc, status hiện tại

## Acceptance Criteria (Kiro)

- AC-1: qe-  
Lệnh generator coverage sinh độc lập, không cần chạy lại generate
- AC-2: Sau mỗi generate (full / incremental), tool tự động cập nhật report nếu file đã tồn tại
- AC-3: Gate KHÔNG tự pass khi % Scenario Coverage < 100% hoặc còn TC Pending review
- AC-4: Thống kê per-unit: số scenario / đã test / % coverage / % pass rate

REQ 30 · PRIORITY: CAO

# Multi-Source Target Scoping

Vấn đề: Khi 2+ mapping cùng đổ vào 1 bảng, TC count gộp cả hai nguồn → kết quả sai

## ❌ Hiện tại — sai với multi-source

```
SELECT COUNT(*)  
FROM silver.target_table  
-- đếm TẤT CẢ nguồn → sai
```

## ✅ Milestone 2 — scope đúng nguồn

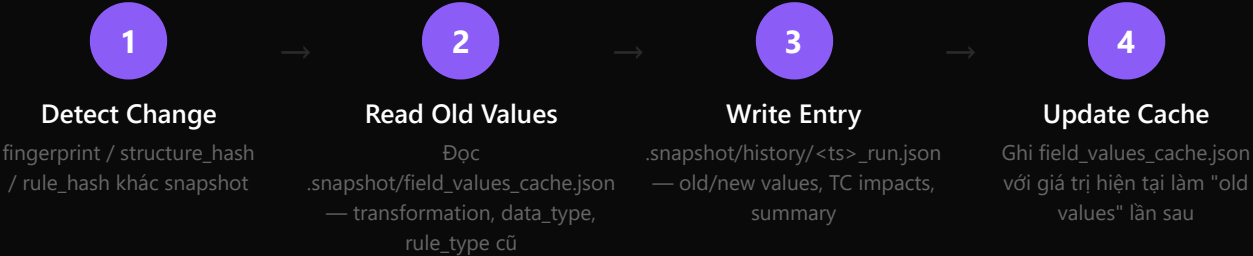
```
SELECT COUNT(*)  
FROM silver.target_table  
WHERE source_system_code = 'NHCK'  
AND data_date = '{{ data_date }}'
```

**AC (Kiro):** Parser nhận diện scope\_columns (source\_system\_code, data\_date) từ mapping\_columns[] · Generator thêm WHERE vào TẤT CẢ SQL template khi unit có scope\_columns · Generator suy luận giá trị scope từ transformation literal (ví dụ 'NHCK') · Warning ⚠️ khi bảng multi-mapping không có scope\_columns · Execution Plan ghi rõ scope\_columns và giá trị per TC



REQ 24 + 25 · PRIORITY: TRUNG BÌNH

# Change History — Audit Trail



**AC chính (Kiro):** Chỉ ghi khi chạy --changed-only và có thay đổi · field\_change chứa old/new value dạng human-readable (không phải hash) · Lỗi ghi history KHÔNG dừng pipeline chính · Auto-cleanup khi > 100 files ·

**CLI history (Req 25):** history — danh sách mới nhất trước · history --detail <run\_id> — old/new values + TC impacts · history --last N · Lần đầu generate → ghi "initial generation" entry

REQ 17+ &amp; 27 · PRIORITY: TRUNG BÌNH

# CLI mới & Dependency Graph

## Lệnh CLI bổ sung (Req 17+)

- `qe-generator diff <file>` — so sánh mapping hiện tại với snapshot, hiển thị field-level changes rõ ràng
- `qe-generator run --changed-only` — dùng Incremental Generator; rollback snapshot nếu render thất bại
- `qe-generator aggregate-report` — gom execution\_result per-unit → tổng hợp; giữ kết quả mới nhất khi trùng key (unit\_name, test\_case\_id)
- `qe-generator coverage` — sinh coverage\_report.xlsx độc lập

## Dependency Graph (Req 27)

- File `config/dependency.yaml` — `depends_on:`  
 khai mỗi `[unit_a,`  
 báo: unit `unit_b]`  
 có
- Khi unit A thay đổi → tool xác định downstream trực tiếp (cấp 1) và in danh sách kèm lý do
- Flag `--cascade` → downstream đệ quy tất cả cấp, không lặp unit
- Detect circular dependency ( $A \rightarrow B \rightarrow A$ ) → raise lỗi liệt kê vòng lặp. Không có file config → tiếp tục bình thường, không báo lỗi

## MILESTONE 2 · SUMMARY

# 6 nhóm requirement cần bổ sung



## Req 29 — Storage Pattern

N+4+M+P TC. SCD4A +10,  
SCD2 +15, Fact +9/8.  
Precondition + No Data logic.  
**Ưu tiên cao nhất.**



## Req 28 — Coverage Report

4-sheet Excel. Exit criteria gate  
Generate→Execute. CLI  
coverage. Auto-update sau  
generate. **Ưu tiên cao.**



## Req 30 — Multi-Source

scope\_columns parser. WHERE  
clause tất cả template.  
Warning thiếu scope. Plan ghi  
scope per TC. **Ưu tiên cao.**



## Req 24/25 — Change History

field\_values\_cache.json. History  
entry JSON. CLI history --  
detail. Lỗi ghi không dừng  
pipeline.



## Req 17+ — CLI mới

diff, --changed-only,  
aggregate-report, coverage.  
Incremental rollback khi  
render lỗi.



## Req 27 — Dependency Graph

config/dependency.yaml.  
Downstream detection. --  
cascade đệ quy. Circular  
dependency check.

Req 29 → 28 → 30 ưu tiên cao · Req 24/25 → 17+ → 27 sau khi nhóm ưu tiên ổn định